

End-to-end flow v0.1: alignment → calibration → grid → capture → spectra → plots

Author: Felipe Fridman G. (ffridman@gmail.com)

This document walks through the full observing pipeline with a narrative tone so new operators can understand not only the "how" but the "why" behind each step. Paths, key flags, and technologies are spelled out so you can trace outputs and logs from start to finish. Built to bring the amateur telescope "Almita" (90 cm grid parabolic antenna + LNA + SDR + OnStep mount) into operational status.

Technical overview (narrative)

We stitch together INDI-based telescope control with rtl_tcp IQ capture and a three-point Y-factor calibration so every spectrum is expressed in physical brightness temperature. IQ and spectra are stored in HDF5 for repeatable, lossless processing; mosaics live in CSV/PNG for quick inspection. NumPy handles FFTs and calibration math, Matplotlib renders sky products, and h5py/JSON keep metadata close to the data. Each stage writes into `data/...` with timestamped session folders, making the pipeline reproducible and traceable across observing nights.

1. Alignment (peak pointing)

- Script: `alignment.py`
- Goal: steer toward peak HI or beacon power before committing long captures.
- Typical command: `python3 alignment.py --radius 0.5 --capture-time 1.5 --iterations 3 --min-elev 20 --host --port 7624 --device "Telescope Simulator" --sdr-host --sdr-port 1234 [--no\_calib] [--no\_sync]`
- Key flags:
 - `--radius`: initial scan radius (deg).
 - `--iterations`: refinement loops to maximize signal.
 - `--capture-time`: seconds per probe.
 - `--no\_calib`: skip applying prior calibration to the metric.
 - `--no\_sync`: skip time sync with INDI.
- Tech: INDI control; rtl_tcp IQ snapshots; NumPy for quick metrics.
- Output: `data/alignment/alignment\_results.json`; console logs (pipe to a file if needed).

2. Calibration (three-point HOT/COLD/LOAD)

- Script: `calibrate.py`
- Goal: reference temperatures to convert power → Kelvin via Y-factor.
- Typical command: `python3 calibrate.py --host --port 7624 --device "Telescope Simulator" --sdr-host --sdr-port 1234 --capture-time 1.5 [--output-base data/calibration]`
- Key flags:
 - `--capture-time`: seconds per point (hot/cold/load).
 - `--output-base`: base folder (default `data/calibration`).
 - `--hot-ra/--hot-dec`, `--cold-ra/--cold-dec`: override catalog targets.

- Tech: INDI, RTL-SDR, h5py, NumPy.
- Output: `data/calibration//` with `calibration\_results.json`, `hot\_calibration.h5`, `cold\_calibration.h5`, `load\_calibration.h5`; logs via console (use `| tee ../calibrate.log`).

3. Grid (mosaic planning)

- Script: `grid\_generator.py`
- Goal: produce RA/DEC waypoints for the mosaic.
- Typical command: `python3 grid\_generator.py --config examples/grid\_example\_cygnus.json [--base-dir data/mosaic]`
- Key flags:
 - `--config`: JSON with center, step, size, projection.
 - `--base-dir`: output root (default `data/mosaic`).
- Tech: NumPy; Matplotlib for PNG; CSV for interoperability.
- Output: `data/mosaic/-YYYYMMDD-HH:MM:SS/{mosaic.csv,mosaic.png}`; optional `grid.log`.

4. Capture (IQ per point)

- Script: `capture.py`
- Goal: record raw IQ at each grid position or single target.
- Typical single-target: `python3 capture.py --host --port 7624 --device "Telescope Simulator" --sdr-host --sdr-port 1234 --duration 10 --target "Cygnus" --ra 20.0 --dec 40.0 [--session-name CygnusTest]`
- Typical grid-run: `python3 capture.py --grid data/mosaic//mosaic.csv --duration 10 --host --port 7624 --device "Telescope Simulator" --sdr-host --sdr-port 1234 [--session-name CygnusGrid]`
- Key flags:
 - `--duration`: seconds per point.
 - `--grid`: CSV from grid_generator.
 - `--session-name`: human-friendly prefix.
- Tech: INDI, RTL-SDR (rtl_tcp/pyrtlsdr), h5py.
- Output: `data/iq/-YYYYMMDD-HH:MM:SS/` with one `.h5` per point (`capture\_0001.h5`, ...); logs via console (optionally `capture.log`).

5. Spectra processing (IQ → Tb)

- Script: `process\_spectra.py`
- Goal: apply three-point calibration and emit brightness-temperature spectra.
- Typical command: `python3 process\_spectra.py --data data/iq// --calibration data/calibration// [--output data/iq//spectrum/] [--fft-size 8192]`
- Key flags:
 - `--fft-size`: spectral resolution; higher is finer but slower.
 - `--output`: destination (default `data/iq//spectrum/`).
- Tech: h5py, NumPy/FFT, JSON metadata.

- Output: ``data/iq//spectrum/`` with ``*_spectrum.h5`` (datasets ``frequencies_hz``, ``tb_kelvin`` plus capture/calibration attrs); logs via console (optional ``process.log``).

6. Plots and maps

- Spectra summary: ``analyze_spectra.py``
- Command: ``python3 analyze_spectra.py --spectra data/iq//spectrum/ --calibration data/calibration/``
- Output: console stats; redirect to ``analyze.log`` if desired.
- Sky map: ``plot_sky_map.py``
- Command: ``python3 plot_sky_map.py --input data/iq//spectrum/ --output-base data/iq//spectrum/ --mode both``
- Outputs: in ``/spectrum/`` → ``plot_scatter.png``, ``plot_interpolated.png``, ``plot_both.png``.
- Modifiers: ``--mode`` (scatter/interpolated/both), ``--limit`` (dB clipping), ``--colormap``, ``--ra-center/--dec-center``.

INDI control helper (GOTO, SYNC, TRACK, connect)

- Class: ``INDITelescopeControl`` in ``indi_telescope_control.py`` (async, XML over TCP).
 - Operations:
 - ``connect()``: opens TCP to INDI, sends ````, issues ``CONNECTION`` switch with `CONNECT=On`.
 - ``goto(ra_hours, dec_degrees)``: sets ``ON_COORD_SET=SLEW``, writes ``EQUATORIAL_EOD_COORD``; waits for ``state="Ok"``.
 - ``sync(ra_real, dec_real)``: toggles ``ON_COORD_SET=SYNC``, sends corrected ``EQUATORIAL_EOD_COORD``, restores TRACK; no physical motion.
 - ``set_tracking(enable)``: flips ``TELESCOPE_TRACK_STATE`` (TRACK_ON/OFF) and logs typical uses; also explains `ON_COORD_SET` modes TRACK/SLEW/SYNC.
 - Logging: timestamped console via ``log()``; set ``verbose=True`` to print the INDI XML primers and byte-level chunks.
 - Usage pattern (sync then goto):
- 1) ``await connect()``; 2) ``await sync(...)`` after plate solving; 3) ``await goto(...)``; 4) ``await set_tracking(True)`` to keep target centered.

Paths and logs at a glance

- Alignment: ``data/alignment//alignment_results.json``
- Calibration: ``data/calibration/{calibration_results.json, hot_*.h5, cold_*.h5, load_*.h5}``
- Grid: ``data/mosaic/-YYYYMMDD-HH:MM:SS/{mosaic.csv,mosaic.png}``
- Raw IQ: ``data/iq/-YYYYMMDD-HH:MM:SS/*.h5``
- Calibrated spectra: ``data/iq//spectrum/*_spectrum.h5``
- Plots: same ``output-base`` (usually ``data/iq//spectrum/``)
- Logs: printed to console; persist with ``| tee /stage.log`` per step.

HDF5 metadata (what each file carries)

- Capture H5 (IQ per point): datasets `i\_samples`, `q\_samples`; attrs like `target\_ra\_hours`, `target\_dec\_degrees`, `sample\_rate\_hz`, `center\_freq\_hz`, `gain\_db`, `timestamp\_iso`, `session\_name`, `sequence\_id`, plus any grid identifiers; used later by processing.
- Calibration H5 (hot/cold/load): datasets `i\_samples`, `q\_samples`; attrs include `capture\_time\_s`, `center\_freq\_hz`, `sample\_rate\_hz`, `gain\_db`, and point metadata (`point\_ra\_hours`, `point\_dec\_degrees`, `tb\_expected\_kelvin`). The companion `calibration\_results.json` records `session\_id`, timestamps, and expected Tb per point.
- Spectrum H5 (`\*\_spectrum.h5`): datasets `frequencies\_hz`, `tb\_kelvin`; attrs propagate capture metadata (target RA/DEC, frequency, sample rate, gain), plus processing info (`fft\_size`, `n\_samples\_processed`, `calibration\_session`, `processing\_timestamp`). This makes spectra self-describing and traceable to the calibration set.

Good practice

- Keep time in sync between PC and mount (`--no\_sync` skips that; only use if already synced).
- Choose a readable `--session-name`; a timestamp is added automatically.
- Save logs with `tee` to debug later.
- Reuse the newest calibration that matches the sky/hardware of your capture session.

Outstanding for v0.1

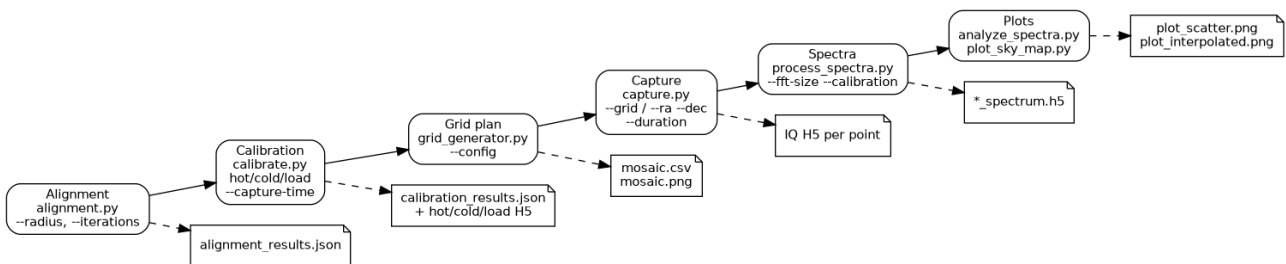
- Session manifest/metadata file (index with paths, versions, commands).

Based on the current setup; adjust flags if you swap INDI/SDR backends.

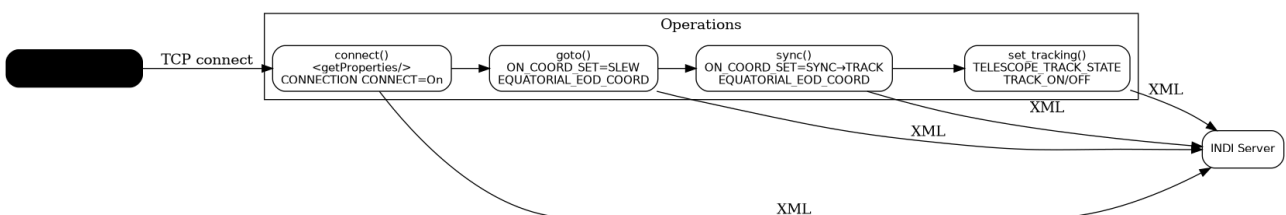
Visual diagrams (Mermaid)

Visual diagrams

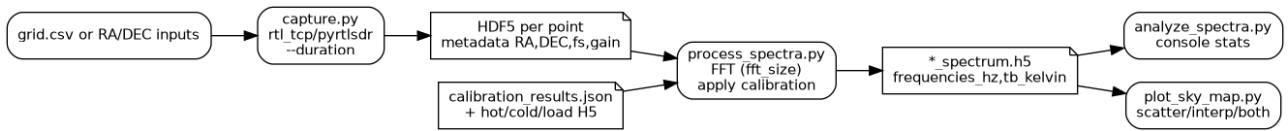
Pipeline overview:



INDITelescopeControl core ops:



Capture → calibrated spectra → plots:



Grid + alignment + calibration:

